Seeding, Reproducibility and Implementation Notes

Companion document to the MRes report "Learning the Quantum Dynamics of Classical Shadows" (MLBD_2025_10)

Vageesh Singh
Imperial College London

August 2026

This document records the code-facing methodology behind the report's results: how randomness is assigned, the design choices it reflects, the audits applied to the estimation pipeline, and the implementation details of the final estimator. It accompanies the code in this repository; the report itself carries the scientific results and the theory.

1 Seeding as experimental design

The report's comparisons rest on error bars computed across random seeds, so how randomness is assigned across conditions is itself a design decision. Three natural implementations bias the error bars even when each individual run is correct.

Nested budget draws. To study how accuracy depends on the number of shadows, one sweeps the shadow count $N_1 < N_2 < \cdots < N_{\max}$. A tempting implementation draws a single stream of $N_{\max}$ snapshots once and reads its first N_i as the estimate at budget N_i. The estimates are then nested: the dataset at N_1 is a subset of that at N_2, so their errors are positively correlated and much of the fluctuation that should distinguish the budgets is common to all of them. An error bar computed across such nested draws measures the variation of a shared prefix, not the independent variability of each budget, and is systematically too small. In the audit that motivated this design, nested prefixes understated the standard error of a budget difference by roughly a quarter (0.0038 against 0.0050), inflating an apparent significance from 16.5σ to 20.3σ. Every budget point in every seed therefore draws an independent dataset.

Globally seeded inference library. Setting the seed of the tensor library once, at the start of a run, ties every subsequent draw, including the initialisation of each Gaussian-process fit, to the order in which the models happen to execute. Re-ordering the sweep, or inserting an extra condition, then silently changes every result downstream. The random state of the inference library is therefore reset per experimental cell, from that cell's own seed, so each configuration is reproducible in isolation and independent of its neighbours.

Additive per-condition seeds. Forming a per-cell seed by adding small offsets to a base seed (base + condition index + repeat index) lets distinct cells land on the same value, secretly correlating conditions that should be independent: for example base $+ 100000 \times$ repeat $+ 1000 \times$ budget collides whenever distinct settings of its terms sum equally.

2 The cell-hash scheme

Each cell of every sweep is described by the tuple (base seed, observable, shadow budget, number of observed times, repeat index). The tuple is passed through numpy's `SeedSequence` to produce a well-mixed 32-bit seed unique to the cell (`make_cell_seed` in `Synthetic_Error_Uncertainty_Check.py`).

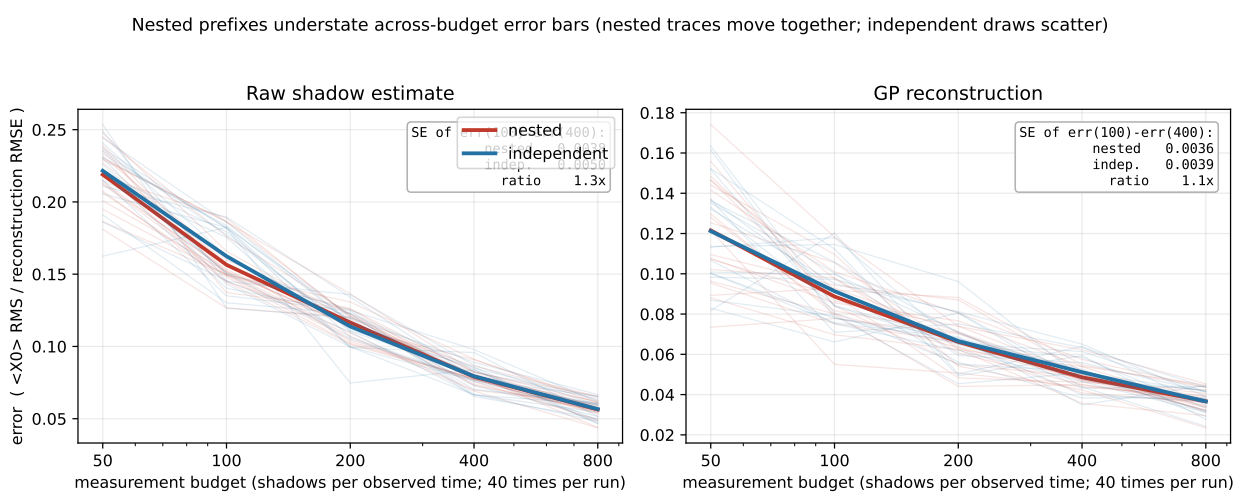

Figure 1: Nested versus independent seeding for a shadow-budget sweep of $\langle X_0 \rangle$. Each faint line is one seed: nested prefixes (red) move together because they share draws, while independent draws (blue) scatter. The inset gives the standard error of the budget contrast $\mathrm{err}(100) - \mathrm{err}(400)$, which nested seeding understates. Left, at the raw-estimate level; right, after Gaussian-process reconstruction.

No two cells share a seed; every cell is reproducible in isolation; both numpy's and torch's random states are set from the cell seed.

3 Audits

Two audits complement the scheme. A *randomness audit* traced every random draw in the codebase to verify that the datasets behind each reported error bar are independent across seeds, that resampled datasets used for uncertainty bands are independent draws rather than reuses, and that no comparison mixes nested with independent sampling. A *leakage audit* traced every use of the exactly simulated dynamics and confirmed the true state enters in exactly two roles: as the simulator from which measurement outcomes are sampled, and in the final error computation. In particular the empirical noise variance used by the fixed-noise reconstructions is computed from the scatter of the measured snapshots alone; the analytic variance expression is never evaluated inside the estimation path, since it would require the unknown expectation value it is being used to estimate.

4 Estimator implementation notes

Matched-count normalisation. Per-time Pauli-string estimates divide by the realised number of basis-aligned snapshots rather than its expectation $N/3^k$; under a fixed-basis protocol the expectation-normalised estimator is scaled by the frozen alignment count for the whole trajectory, a bias no smoother can remove.

Smoothed variance for small counts. With few aligned snapshots the raw sample variance of the binary outcome products is frequently zero, which would tell the regression those points are noise-free. A Laplace-smoothed binomial estimate, $\widehat{\mathrm{se}}^2 = 4p_s(1-p_s)/m$ with $p_s = (n_+ + 1)/(m+2)$, keeps the noise input well defined.

Exact pattern-count sampling. For simulation studies the per-snapshot measurement loop is replaced by exact multinomials: pattern counts over the 3^n basis patterns, then outcome counts from the Born distribution within each pattern. This is statistically identical to per-snapshot simulation for every estimator used, and orders of magnitude faster. Outcome-bit ordering follows the tensor

(kron) convention, with qubit 0 as the highest bit, and is checked by the large-N validation gate below.

Validation gates. New estimator code is validated at large N against the exact state (inputs must approach truth within the noise floor) before entering any study; scripts carry a `QUICK=1` smoke mode and stamp their configuration, seeds and wall time into their output CSVs.

5 Experimental design considerations

These design considerations are summarised in one paragraph of the report (its Section 3.1); the full rationale is recorded here.

Budget-matched comparison. Every comparison between routes or strategies is made at a fixed measurement budget: the total number of single-shot snapshots is held constant, so a method using more times uses proportionally fewer shots per time. Comparing at equal budget is what makes "breadth versus depth" a fair question rather than a foregone one. Each configuration is run over many independent random seeds, and differences are reported with a $2\times$ standard-error interval on the paired difference; a gap smaller than that is not claimed as an effect. This guards against reading structure into seed-to-seed scatter, the commonest way a favourable single run misleads.

Inputs to the regression. Two choices concern what the regression is given. First, per-time Pauli-string estimates are normalised by the realised number of basis-aligned snapshots rather than by its expectation. The naive alternative leaves every estimate scaled by the alignment count's fluctuation; under a fixed-basis protocol that fluctuation is frozen and shared by the entire trajectory, where no smoother can remove it, since an error common to all points is indistinguishable from signal. Second, the observation-noise variance supplied to the regression is the measured scatter of the snapshots, not a fitted hyperparameter: fitting it by marginal likelihood systematically under-estimates the noise at the dataset sizes used (central ratio 0.68 of the analytic value over twenty seeds), which both narrows the uncertainty bands and lets the posterior mean chase shot noise. When few snapshots align, the raw sample variance of the binary outcome products is frequently zero, which would tell the regression those points are noise-free, so the Laplace-smoothed variance estimate of the implementation notes above keeps the noise input well defined.

The smoothness prior. A Gaussian-process prior that is too strong flattens real structure: a lengthscale chosen too long averages across a genuine oscillation and shrinks its amplitude, reporting a confident but attenuated curve. This is the characteristic failure of the regression routes and a real risk for the fast coherences of $\rho(t)$. It is controlled by the marginal-likelihood fit of the lengthscale: an over-long lengthscale that misses an oscillation is penalised by the data it fails to explain. Where the sampling in time is too coarse to resolve a coherence at all, no choice of lengthscale can recover it (a Nyquist limit), but within the resolved band the fitted prior neither over- nor under-smooths by construction.

Software and regeneration. All results come from scripted sweeps that record their seeds, budgets and hyperparameter settings in their output CSVs, so each figure can be regenerated exactly: the study scripts in `studies/` and `estimator_comparison/` produce the CSVs, and the scripts in `figures/` turn them into the report's figures. Quantum dynamics are simulated with QuTiP; the Gaussian-process routes use GPyTorch and BoTorch (fitted-noise comparisons) and scikit-learn (final method and per-element studies).

6 Repository map

- Root: measurement simulation, shadow construction, cell-hash seeding (`Synthetic_Error_Uncertainty_Che`
  classifier-route machinery (`Bayesian_part.py`, `quantum_part.py`), shadow-matrix utilities.
- `rho_reconstruction/`: joint shadow matrices and the conditional (autoregressive) route.
- `studies/`: the final-method studies reported in the MRes report: route comparisons (`routes3_final.py`), sensitivity ladder (`final_config_coverage.py`), headline reconstruction (`final_headline_recon.py`), the $\rho(t)$ program (`rho_final_program.py`, `partial_final.py`), budget allocation (`budget_final.py`), the Hamiltonian sweep recompute (`ham_gp_recompute.py`), and the diagnostic studies (coverage ablations, median-of-means, classical-smoother baselines). Summary CSVs of each study's results sit alongside the scripts.